

Based on the inventory system project demonstrated in the video, here is a practical, university-level exam question designed to test your understanding of Java OOP principles.

Computer Science 201: Object-Oriented Programming Exam

Time Allowed: 45 Minutes

Topic: Inheritance, Encapsulation, Polymorphism, and Abstraction

Scenario:

You are developing a **Video Game Inventory System**. The system needs to handle different types of items that a player can collect. Every item has a name and a quantity. Some items are "Consumables" (like fruit) which have a specific variety, and some are "Equippables" (like weapons) which have damage values and combat types.

Question 1: Encapsulation & Abstraction (10 Marks)

1. Define an **abstract class** named Item.
2. It must contain two private fields: name (String) and quantity (int).
3. Implement a **constructor** to initialize these fields.
4. Provide **getter methods** for both fields.
5. Declare an **abstract method** called displayInfo() that returns void.

Question 2: Inheritance & Method Overriding (15 Marks)

Create a subclass of Item named Weapon.

1. Add private fields: damage (int) and weaponType (String) (e.g., "Melee", "Ranged").
2. Implement a constructor that uses the super keyword to initialize the name and quantity from the parent class.
3. **Override** the displayInfo() method to print the following format: Weapon: [name] | Quantity: [qty] | Damage: [dmg] | Type: [type]

Question 3: Polymorphism & Collection Management (15 Marks)

Create a class named Inventory.

1. Use a private ArrayList<Item> to store the items.
2. Implement a method addItem(Item newItem) that adds an item to the list.
3. Implement a method showAllItems() that iterates through the list and calls displayInfo() on each object.
4. **Bonus (Polymorphic Overloading):** Create an overloaded version of addItem that takes (String name, int qty, int dmg, String type) as parameters, creates a Weapon object internally, and adds it to the list.

Question 4: Analysis (5 Marks)

In the Inventory class, we store Weapon objects inside an ArrayList of type Item.

- **A)** What type of polymorphism is being demonstrated when we call `item.displayInfo()` inside the loop?
 - **B)** Why was the Item class made abstract instead of a regular class?
-

Implementation Hint (From the Video):

- **Encapsulation:** Ensure your fields are private and accessed via methods [\[02:13\]](#).
 - **Super Keyword:** Remember that `super()` must be the first line in your subclass constructor [\[09:13\]](#).
 - **Abstraction:** An abstract class cannot be instantiated directly; it serves as a blueprint [\[22:33\]](#).
-

Practical Task:

Write the Java code for the classes described above and create a Main class to test adding a "Steel Sword" (Quantity: 1, Damage: 50, Type: "Melee") to the inventory and displaying it.